

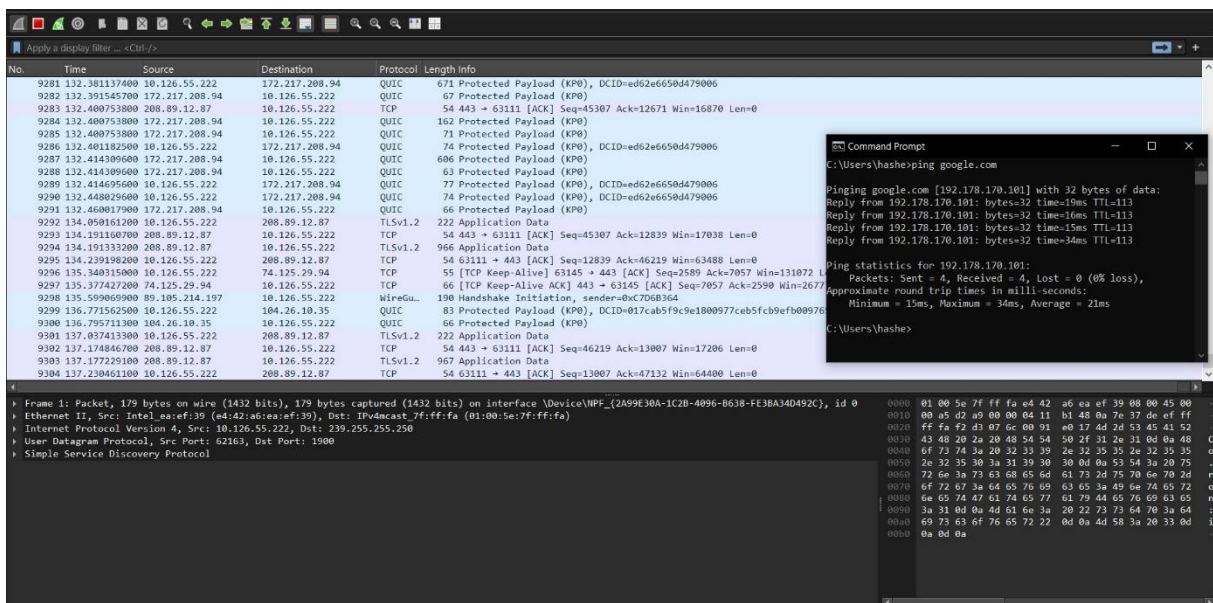
Network Protocol Analysis and Performance Diagnostic Report

Introduction

This report documents a technical investigation into the network infrastructure of the University of the People Uo People following reports of persistent network latency. As a junior network engineer, I utilized Wireshark to capture and analyze real-time traffic, aiming to identify the protocols in operation and determine their impact on performance. According to Sharpe et al. (n.d.), gaining visibility into data flows is essential for diagnosing bottlenecks. By capturing and filtering real network traffic, I was able to observe exactly how data flows across the organization's network to identify potential issues.

Methodology

The diagnostic process involved capturing live packets on the WLAN interface while interacting with the Uo People student portal. To verify external connectivity, I executed a Ping test to google.com. Conceptually, a ping test works like knocking on a friend's door and calling his name; if he answers, you know he is "home" and active. In network terms, the successful reply from Google confirmed that the connection was functional, showing 0% packet loss (ChemiCloud, 2020). After the capture, I applied specific filters to study the protocols in detail.



Protocol Identification

During the session, I identified several protocols essential for communication and security.

Below are five specific protocols observed:

- **WireGuard (VPN Protocol):** Observed in my capture. WireGuard is a modern protocol for secure data encapsulation. The Handshake Initiation represents the initial handshake required to open a secure tunnel (Angelo, 2019).
- **TLSv1.3 (Transport Layer Security):** This acts like a sealed envelope for data, ensuring that the application data remains encrypted and private. TLS 1.3 is the latest version, offering faster handshakes.
- **DNS (Domain Name System):** DNS works like a phonebook for the internet, translating human-readable names like google.com into IP addresses that devices can understand.
- **QUIC (Quick UDP Internet Connections):** QUIC is designed for speed, reducing latency by streamlining the connection process between the client and the server.
- **TCP (Transmission Control Protocol):** The backbone of reliable data transfer, ensuring that all web portal elements are delivered without errors.

505	8.084458600	142.251.154.119	10.126.55.222	QUIC	66 Protected Payload (KP0)
506	8.108026600	10.126.55.222	10.126.55.140	DNS	74 Standard query 0x328b HTTPS lh3.google.com
507	8.108458000	10.126.55.222	10.126.55.140	DNS	74 Standard query 0xec6d A lh3.google.com
508	8.126692800	10.126.55.140	10.126.55.222	DNS	144 Standard query response 0x328b HTTPS lh3.google.com CNAME lh2.l.google.com SOA ns1.google.com
509	8.126692800	10.126.55.140	10.126.55.222	DNS	190 Standard query response 0xec6d A lh3.google.com CNAME lh2.l.google.com A 74.125.29.138 A 74.125.29.113 A 74.125.29.101 A 74.125.29.102
510	8.128419900	10.126.55.222	74.125.29.138	TCP	66 63061 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
511	8.142637200	74.125.29.138	10.126.55.222	TCP	66 443 → 63061 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
512	8.142743300	10.126.55.222	74.125.29.138	TCP	54 63061 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=0
513	8.143413200	10.126.55.222	74.125.29.138	TCP	1466 63061 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=1412 [TCP PDU reassembled in 514]
514	8.143413200	10.126.55.222	74.125.29.138	TLSv1.3	462 Client Hello (SHI=lh3.google.com)
515	8.161668400	74.125.29.138	10.126.55.222	TCP	54 443 → 63061 [ACK] Seq=1 Ack=1413 Win=268288 Len=0
516	8.166678400	74.125.29.138	10.126.55.222	TCP	54 443 → 63061 [ACK] Seq=1 Ack=1821 Win=268032 Len=0
517	8.184593300	74.125.29.138	10.126.55.222	TLSv1.3	4298 Server Hello, Change Cipher Spec
518	8.184593300	74.125.29.138	10.126.55.222	TCP	873 [TCP Previous segment not captured] 443 → 63061 [PSH, ACK] Seq=7861 Ack=1821 Win=268032 Len=819 [TCP PDU reassembled in 522]
519	8.184593300	74.125.29.138	10.126.55.222	TCP	1466 [TCP Retransmission] 443 → 63061 [PSH, ACK] Seq=4237 Ack=1821 Win=268032 Len=1412 [TCP PDU reassembled in 522]
520	8.185081500	10.126.55.222	74.125.29.138	TCP	66 63061 → 443 [ACK] Seq=1821 Ack=4237 Win=131072 Len=0 SLE=7061 SRE=7880
521	8.185337400	10.126.55.222	74.125.29.138	TCP	66 63061 → 443 [ACK] Seq=1821 Ack=5649 Win=131072 Len=0 SLE=7061 SRE=7880
522	8.185437800	74.125.29.138	10.126.55.222	TLSv1.3	1466 [TCP Out-Of-Order] , Application Data
523	8.185466500	10.126.55.222	74.125.29.138	TCP	54 63061 → 443 [ACK] Seq=1821 Ack=7880 Win=131072 Len=0
524	8.188474100	10.126.55.222	74.125.29.138	TLSv1.3	128 Change Cipher Spec, Application Data
525	8.188877800	10.126.55.222	74.125.29.138	TLSv1.3	146 Application Data
526	8.189292400	10.126.55.222	74.125.29.138	TCP	1466 63061 → 443 [ACK] Seq=1987 Ack=7880 Win=131072 Len=1412 [TCP PDU reassembled in 528]
527	8.189292400	10.126.55.222	74.125.29.138	TCP	1466 63061 → 443 [ACK] Seq=3399 Ack=7880 Win=131072 Len=1412 [TCP PDU reassembled in 528]
528	8.189292400	10.126.55.222	74.125.29.138	TLSv1.3	141 Application Data

13	2.990498000	10.126.55.222	50.85.242.216	TCP	1478	63046 → 443 [ACK] Seq=378 Ack=8582 Win=132352 Len=1424 [TCP PDU reassembled in 14]
14	2.990498000	10.126.55.222	50.85.242.216	TLSv1.2	508	Application Data
15	2.991128600	10.126.55.222	50.85.242.216	TCP	1478	63046 → 443 [ACK] Seq=2256 Ack=8582 Win=132352 Len=1424 [TCP PDU reassembled in 16]
16	2.991128600	10.126.55.222	50.85.242.216	TLSv1.2	449	Application Data
17	3.030178000	50.85.242.216	10.126.55.222	TCP	54	443 → 63046 [ACK] Seq=8582 Ack=2256 Win=4195072 Len=0
18	3.032118400	50.85.242.216	10.126.55.222	TCP	54	443 → 63046 [ACK] Seq=8582 Ack=4075 Win=4195072 Len=0
19	3.033018400	50.85.242.216	10.126.55.222	TLSv1.2	87	Encrypted Handshake Message
20	3.033857000	10.126.55.222	50.85.242.216	TLSv1.2	277	Encrypted Handshake Message
21	3.072963600	50.85.242.216	10.126.55.222	TCP	5750	443 → 63046 [ACK] Seq=8615 Ack=4298 Win=4194816 Len=5696 [TCP PDU reassembled in 24]
22	3.072963600	50.85.242.216	10.126.55.222	TCP	124	[TCP Previous segment not captured] 443 → 63046 [PSH, ACK] Seq=17159 Ack=4298 Win=4194816 Len=70 [TCP PDU reassembled in 24]
23	3.073141800	10.126.55.222	50.85.242.216	TCP	66	63046 → 443 [ACK] Seq=4298 Ack=14311 Win=132352 Len=0 SLE=17159 SRE=17229
24	3.074435300	50.85.242.216	10.126.55.222	TLSv1.2	2902	[TCP Out-Of-Order] , Encrypted Handshake Message
25	3.074613100	10.126.55.222	50.85.242.216	TCP	54	63046 → 443 [ACK] Seq=4298 Ack=17229 Win=132352 Len=0
26	3.080641500	10.126.55.222	50.85.242.216	TLSv1.2	267	Encrypted Handshake Message, Change Cipher Spec, Encrypted Handshake Message
27	3.112208600	50.85.242.216	10.126.55.222	TLSv1.2	129	Change Cipher Spec, Encrypted Handshake Message
28	3.114198000	50.85.242.216	10.126.55.222	TCP	1019	[TCP Previous segment not captured] 443 → 63046 [PSH, ACK] Seq=18728 Ack=4511 Win=4194560 Len=965
29	3.114254400	10.126.55.222	50.85.242.216	TCP	66	63046 → 443 [ACK] Seq=4511 Ack=17304 Win=132352 Len=0 SLE=18728 SRE=19693
30	3.118627500	50.85.242.216	10.126.55.222	TCP	1478	[TCP Out-Of-Order] 443 → 63046 [ACK] Seq=17304 Ack=4511 Win=4194560 Len=1424
31	3.118718200	10.126.55.222	50.85.242.216	TCP	54	63046 → 443 [ACK] Seq=4511 Ack=19693 Win=132352 Len=0
32	3.137012800	10.126.55.222	50.85.242.216	TCP	1478	63046 → 443 [ACK] Seq=4511 Ack=19693 Win=132352 Len=1424 [TCP PDU reassembled in 33]
33	3.137012800	10.126.55.222	50.85.242.216	TLSv1.2	495	Application Data
34	3.172510100	50.85.242.216	10.126.55.222	TCP	54	443 → 63046 [ACK] Seq=19693 Ack=6376 Win=4195072 Len=0
35	3.335802600	50.85.242.216	10.126.55.222	TLSv1.2	563	Application Data

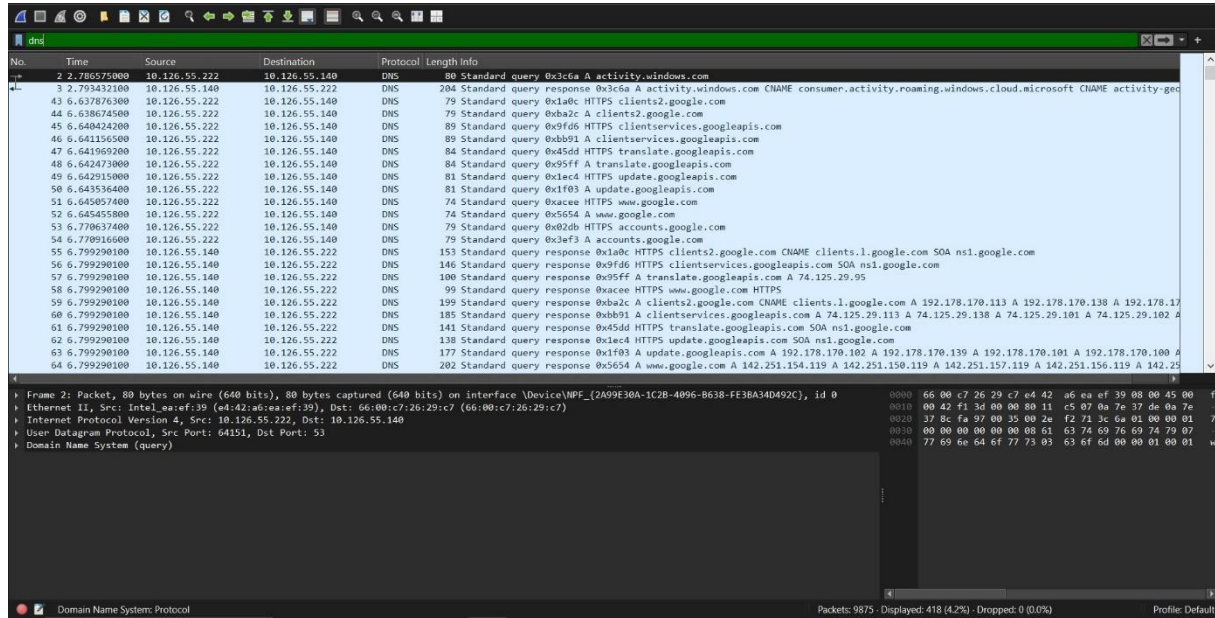
Quantitative Packet Analysis

To investigate the slow network complaint, I quantified the traffic volume for specific protocols using Wireshark’s display filters:

HTTP: Filtering for http revealed 14 packets. These represent the actual web page requests and responses between my device and the server.

The image displays a Wireshark packet capture of HTTP traffic. The top pane shows the packet list with 14 HTTP packets (No. 78-91) between source 10.126.55.222 and destination 192.178.170.113. The middle pane shows the details of packet 78, which is an HTTP GET request for /per/492350f6-3a01-4f97-b9c0-c7c6ddf67d60/Office/Data/v64_16.0.19822.20182.cab. The bottom pane shows the raw packet bytes in hexadecimal and ASCII format.

DNS: Filtering for dns showed 418 packets. Since every website visit requires a DNS query first, a high volume here is expected, though it can cause perceived slowness if the phonebook lookup is delayed



No.	Time	Source	Destination	Protocol	Length	Info
2	2.786575000	10.126.55.222	10.126.55.140	DNS	80	Standard query 0x3d6a A activity.windows.com
3	2.793432100	10.126.55.140	10.126.55.222	DNS	204	Standard query response 0x3d6a A activity.windows.com CNAME consumer.activity.roaming.windows.cloud.microsoft CNAME activity-ged
43	6.637876300	10.126.55.222	10.126.55.140	DNS	79	Standard query 0x1a0c HTTPS clients2.google.com
44	6.638674500	10.126.55.222	10.126.55.140	DNS	79	Standard query 0xba2c A clients2.google.com
45	6.640424200	10.126.55.222	10.126.55.140	DNS	89	Standard query 0x9fd6 HTTPS clientservices.googleapis.com
46	6.641194500	10.126.55.222	10.126.55.140	DNS	89	Standard query 0xd091 A clientservices.googleapis.com
47	6.641969200	10.126.55.222	10.126.55.140	DNS	84	Standard query 0x45dd HTTPS translate.googleapis.com
48	6.642473000	10.126.55.222	10.126.55.140	DNS	84	Standard query 0x95ff A translate.googleapis.com
49	6.642915000	10.126.55.222	10.126.55.140	DNS	81	Standard query 0x1ec4 HTTPS update.googleapis.com
50	6.643536400	10.126.55.222	10.126.55.140	DNS	81	Standard query 0xf03 A update.googleapis.com
51	6.645057400	10.126.55.222	10.126.55.140	DNS	74	Standard query 0xacee HTTPS www.google.com
52	6.645455800	10.126.55.222	10.126.55.140	DNS	74	Standard query 0x5054 A www.google.com
53	6.770637400	10.126.55.222	10.126.55.140	DNS	79	Standard query 0x82db HTTPS accounts.google.com
54	6.770916600	10.126.55.222	10.126.55.140	DNS	79	Standard query 0x3ef3 A accounts.google.com
55	6.799290100	10.126.55.140	10.126.55.222	DNS	153	Standard query response 0x1a0c HTTPS clients2.google.com CNAME clients1.google.com SOA ns1.google.com
56	6.799290100	10.126.55.140	10.126.55.222	DNS	140	Standard query response 0x9fd6 HTTPS clientservices.googleapis.com SOA ns1.google.com
57	6.799290100	10.126.55.140	10.126.55.222	DNS	100	Standard query response 0x95ff A translate.googleapis.com A 74.125.29.95
58	6.799290100	10.126.55.140	10.126.55.222	DNS	99	Standard query response 0xacee HTTPS www.google.com HTTPS
59	6.799290100	10.126.55.222	10.126.55.222	DNS	199	Standard query response 0xba2c A clients2.google.com CNAME clients1.google.com A 192.178.170.113 A 192.178.170.138 A 192.178.170.114
60	6.799290100	10.126.55.140	10.126.55.222	DNS	185	Standard query response 0xd091 A clientservices.googleapis.com A 74.125.29.113 A 74.125.29.138 A 74.125.29.101 A 74.125.29.102
61	6.799290100	10.126.55.140	10.126.55.222	DNS	141	Standard query response 0x45dd HTTPS translate.googleapis.com SOA ns1.google.com
62	6.799290100	10.126.55.140	10.126.55.222	DNS	138	Standard query response 0x1ec4 HTTPS update.googleapis.com SOA ns1.google.com
63	6.799290100	10.126.55.140	10.126.55.222	DNS	177	Standard query response 0xf03 A update.googleapis.com A 192.178.170.102 A 192.178.170.101 A 192.178.170.100 A 192.178.170.104
64	6.799290100	10.126.55.140	10.126.55.222	DNS	202	Standard query response 0x5054 A www.google.com A 142.251.154.119 A 142.251.150.119 A 142.251.157.119 A 142.251.156.119 A 142.251.155.119

Frame 2: Packet, 80 bytes on wire (640 bits), 80 bytes captured (640 bits) on interface \Device\NPF_{2A09E30A-1C2B-4096-B638-FE3BA3D492C}, id 0

Ethernet II, Src: Intel_ea:ef:39 (e4:42:a6:ea:ef:39), Dst: 66:00:c7:26:29:c7 (66:00:c7:26:29:c7)

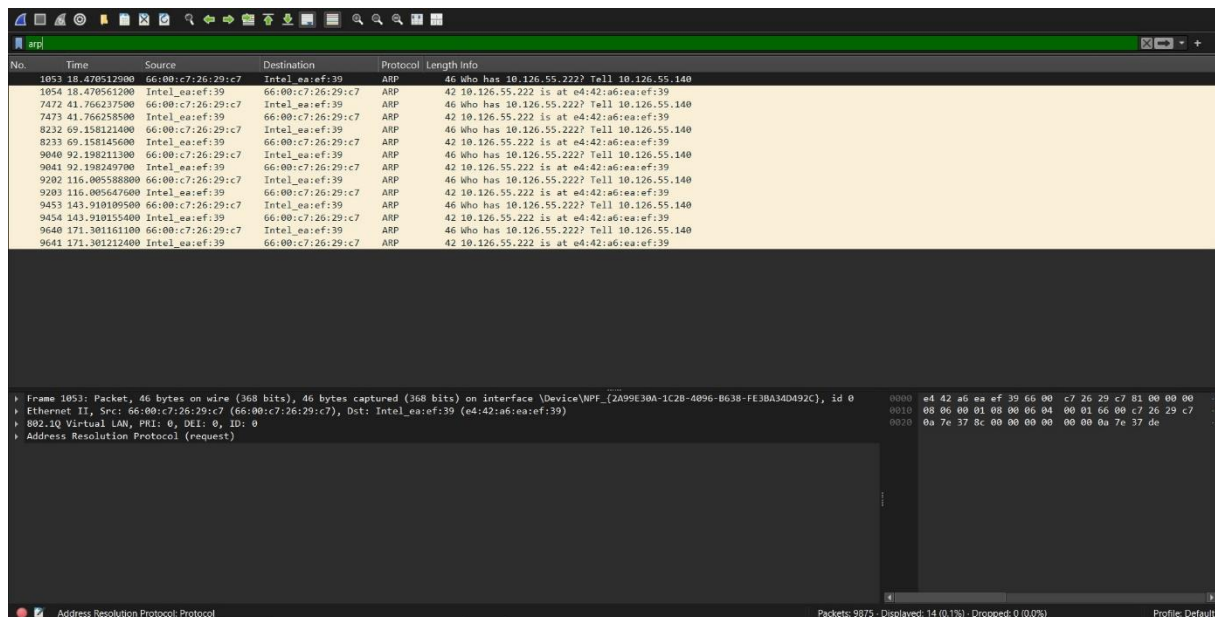
Internet Protocol Version 4, Src: 10.126.55.222, Dst: 10.126.55.140

User Datagram Protocol, Src Port: 64151, Dst Port: 53

Domain Name System (query)

Packets: 9875 - Displayed: 418 (4.2%) - Dropped: 0 (0.0%) Profile: Default

ARP: I observed 14 ARP packets. ARP is responsible for mapping IP addresses to physical MAC addresses, helping devices on the local network identify and communicate with each other



No.	Time	Source	Destination	Protocol	Length	Info
1053	18.470512000	66:00:c7:26:29:c7	Intel_ea:ef:39	ARP	46	Who has 10.126.55.222? Tell 10.126.55.140
1054	18.470561200	Intel_ea:ef:39	66:00:c7:26:29:c7	ARP	42	10.126.55.222 is at e4:42:a6:ea:ef:39
7472	41.766237500	66:00:c7:26:29:c7	Intel_ea:ef:39	ARP	46	Who has 10.126.55.222? Tell 10.126.55.140
7473	41.766258500	Intel_ea:ef:39	66:00:c7:26:29:c7	ARP	42	10.126.55.222 is at e4:42:a6:ea:ef:39
8232	69.158121400	66:00:c7:26:29:c7	Intel_ea:ef:39	ARP	46	Who has 10.126.55.222? Tell 10.126.55.140
8233	69.158145000	Intel_ea:ef:39	66:00:c7:26:29:c7	ARP	42	10.126.55.222 is at e4:42:a6:ea:ef:39
9040	92.198211300	66:00:c7:26:29:c7	Intel_ea:ef:39	ARP	46	Who has 10.126.55.222? Tell 10.126.55.140
9041	92.198249700	Intel_ea:ef:39	66:00:c7:26:29:c7	ARP	42	10.126.55.222 is at e4:42:a6:ea:ef:39
9202	116.005588800	66:00:c7:26:29:c7	Intel_ea:ef:39	ARP	46	Who has 10.126.55.222? Tell 10.126.55.140
9203	116.005647600	Intel_ea:ef:39	66:00:c7:26:29:c7	ARP	42	10.126.55.222 is at e4:42:a6:ea:ef:39
9453	143.910109500	66:00:c7:26:29:c7	Intel_ea:ef:39	ARP	46	Who has 10.126.55.222? Tell 10.126.55.140
9454	143.910155400	Intel_ea:ef:39	66:00:c7:26:29:c7	ARP	42	10.126.55.222 is at e4:42:a6:ea:ef:39
9640	171.301161100	66:00:c7:26:29:c7	Intel_ea:ef:39	ARP	46	Who has 10.126.55.222? Tell 10.126.55.140
9641	171.301212400	Intel_ea:ef:39	66:00:c7:26:29:c7	ARP	42	10.126.55.222 is at e4:42:a6:ea:ef:39

Frame 1053: Packet, 46 bytes on wire (368 bits), 46 bytes captured (368 bits) on interface \Device\NPF_{2A09E30A-1C2B-4096-B638-FE3BA3D492C}, id 0

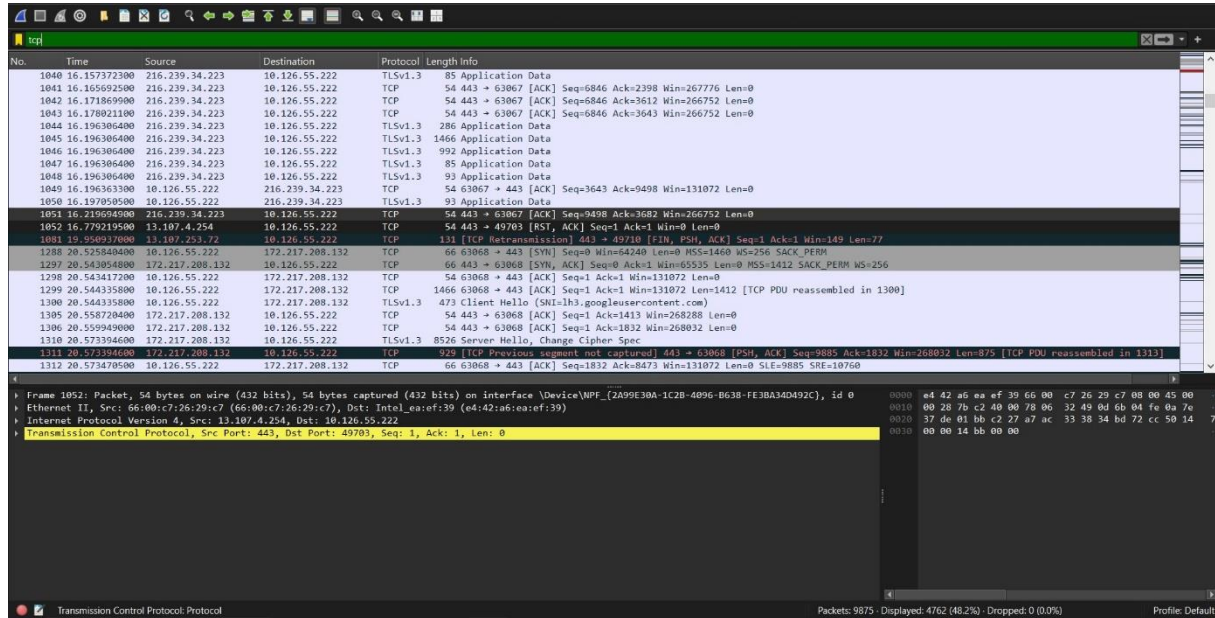
Ethernet II, Src: 66:00:c7:26:29:c7 (66:00:c7:26:29:c7), Dst: Intel_ea:ef:39 (e4:42:a6:ea:ef:39)

802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 0

Address Resolution Protocol (request)

Packets: 9875 - Displayed: 14 (0.1%) - Dropped: 0 (0.0%) Profile: Default

TCP: Filtering for tcp showed 4,762 packets, accounting for 48.2% of the total capture. TCP requires an acknowledgment for every packet sent, which naturally increases the packet count but ensures reliability.

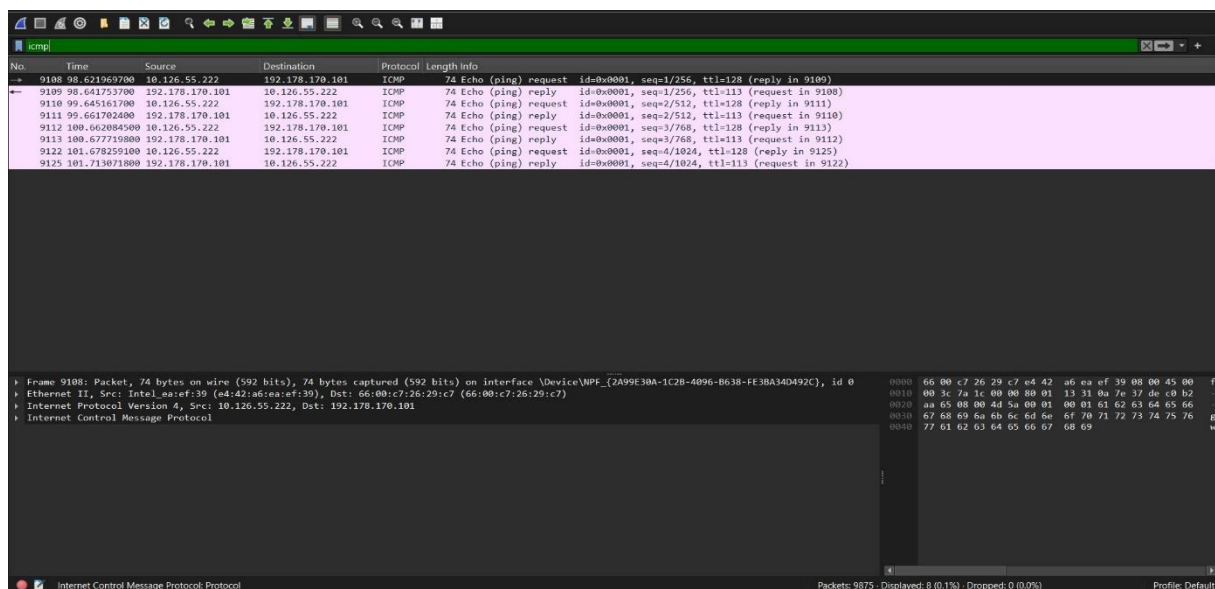


No.	Time	Source	Destination	Protocol	Length	Info
1040	16.157372500	216.239.34.223	10.126.55.222	TLSv1.3	85	Application Data
1041	16.165692500	216.239.34.223	10.126.55.222	TCP	54	443 → 63067 [ACK] Seq=6846 Ack=2398 Win=267776 Len=0
1042	16.171869900	216.239.34.223	10.126.55.222	TCP	54	443 → 63067 [ACK] Seq=6846 Ack=3612 Win=266752 Len=0
1043	16.178021100	216.239.34.223	10.126.55.222	TCP	54	443 → 63067 [ACK] Seq=6846 Ack=3643 Win=266752 Len=0
1044	16.196306400	216.239.34.223	10.126.55.222	TLSv1.3	286	Application Data
1045	16.196306400	216.239.34.223	10.126.55.222	TLSv1.3	1466	Application Data
1046	16.196306400	216.239.34.223	10.126.55.222	TLSv1.3	992	Application Data
1047	16.196306400	216.239.34.223	10.126.55.222	TLSv1.3	85	Application Data
1048	16.196306400	216.239.34.223	10.126.55.222	TLSv1.3	93	Application Data
1049	16.196363300	10.126.55.222	216.239.34.223	TCP	54	63067 → 443 [ACK] Seq=3643 Ack=9498 Win=131072 Len=0
1050	16.197809000	10.126.55.222	216.239.34.223	TLSv1.3	93	Application Data
1051	16.219694900	216.239.34.223	10.126.55.222	TCP	54	443 → 63067 [ACK] Seq=9498 Ack=3682 Win=266752 Len=0
1052	16.779219500	13.107.4.254	10.126.55.222	TCP	54	443 → 49703 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1081	19.950937000	13.107.253.72	10.126.55.222	TCP	131	[TCP Retransmission] 443 → 49710 [FIN, PSH, ACK] Seq=1 Ack=1 Win=149 Len=77
1288	20.525848000	10.126.55.222	172.217.208.132	TCP	66	63068 → 443 [SYN] Seq=0 Win=64248 Len=0 MSS=1460 WS=256 SACK_PERM
1289	20.543054000	172.217.208.132	10.126.55.222	TCP	66	443 → 63068 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
1298	20.543417200	10.126.55.222	172.217.208.132	TCP	54	63068 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=0
1299	20.544335800	10.126.55.222	172.217.208.132	TCP	1466	63068 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=1412 [TCP PDU reassembled in 1300]
1300	20.544335800	10.126.55.222	172.217.208.132	TLSv1.3	473	Client Hello (SHA1=1h3.gongleusercontent.com)
1305	20.558720400	172.217.208.132	10.126.55.222	TCP	54	443 → 63068 [ACK] Seq=1 Ack=1413 Win=268288 Len=0
1306	20.559949000	172.217.208.132	10.126.55.222	TCP	54	443 → 63068 [ACK] Seq=1 Ack=1832 Win=268832 Len=0
1310	20.573394600	172.217.208.132	10.126.55.222	TLSv1.3	8526	Server Hello, Change Cipher Spec
1311	20.573394600	172.217.208.132	10.126.55.222	TCP	929	[TCP Previous segment not captured] 443 → 63068 [PSH, ACK] Seq=9885 Ack=1832 Win=268832 Len=875 [TCP PDU reassembled in 1313]
1312	20.573470500	10.126.55.222	172.217.208.132	TCP	66	63068 → 443 [ACK] Seq=1832 Ack=8473 Win=131072 Len=0 SLE=9885 SRE=10760

From 1052: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF_{2A99E30A-1C2B-4096-B638-FE3BA3D492C}, id 0
Ethernet II, Src: Intel_ea:ef:c7:26:29:c7, Dst: Intel_ea:ef:c7:26:29:c7 (ea:42:a6:ea:ef:c7)
Internet Protocol Version 4, Src: 13.107.4.254, Dst: 10.126.55.222
Transmission Control Protocol, Src Port: 443, Dst Port: 49703, Seq: 1, Ack: 1, Len: 0

Packets: 9875 · Displayed: 4762 (48.2%) · Dropped: 0 (0.0%) Profile: Default

ICMP: I captured 8 packets directly from the ping test, confirming stable connectivity with 4 requests and 4 replies.



No.	Time	Source	Destination	Protocol	Length	Info
9108	98.621969700	10.126.55.222	192.178.170.101	ICMP	74	Echo (ping) request id=0x0001, seq=1/256, ttl=128 (reply in 9109)
9109	98.641753700	192.178.170.101	10.126.55.222	ICMP	74	Echo (ping) reply id=0x0001, seq=1/256, ttl=113 (request in 9108)
9110	99.665161700	10.126.55.222	192.178.170.101	ICMP	74	Echo (ping) request id=0x0001, seq=2/512, ttl=128 (reply in 9111)
9111	99.661702400	192.178.170.101	10.126.55.222	ICMP	74	Echo (ping) reply id=0x0001, seq=2/512, ttl=113 (request in 9110)
9112	100.062084500	10.126.55.222	192.178.170.101	ICMP	74	Echo (ping) request id=0x0001, seq=3/768, ttl=128 (reply in 9113)
9113	100.677719800	192.178.170.101	10.126.55.222	ICMP	74	Echo (ping) reply id=0x0001, seq=3/768, ttl=113 (request in 9112)
9122	101.678259100	10.126.55.222	192.178.170.101	ICMP	74	Echo (ping) request id=0x0001, seq=4/1024, ttl=128 (reply in 9125)
9125	101.713071800	192.178.170.101	10.126.55.222	ICMP	74	Echo (ping) reply id=0x0001, seq=4/1024, ttl=113 (request in 9122)

From 9108: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface \Device\NPF_{2A99E30A-1C2B-4096-B638-FE3BA3D492C}, id 0
Ethernet II, Src: Intel_ea:ef:c7:26:29:c7, Dst: 66:80:c7:26:29:c7 (66:00:c7:26:29:c7)
Internet Protocol Version 4, Src: 10.126.55.222, Dst: 192.178.170.101
Internet Control Message Protocol

Packets: 9875 · Displayed: 8 (0.1%) · Dropped: 0 (0.0%) Profile: Default

Conclusion

The analysis shows a healthy variety of protocols. The high volume of TCP packets suggests that most of the network activity was dedicated to reliable data transmission. However, the "slow network" complaint may be related to how traffic is prioritized. As Swyx (2020b) suggests, effective Traffic Engineering is required to manage these flows. Furthermore, the use of VPNs (WireGuard), while providing security, adds an extra layer of encapsulation that can impact throughput (Jadhav & Sheth, 2021). Tools like Wireshark are essential for any network engineer to understand what is happening inside a network and resolve performance issues.

References

- Angelo, R. (2019). Secure protocols and virtual private networks: An evaluation. *Issues in Information Systems*, 20(3), 37-46.
- ChemiCloud. (2020, August 9). *How to perform a ping test in windows, mac OS, and linux*. <https://chemicloud.com/kb/article/how-to-perform-a-ping-test/>
- Jadhav, R. R., & Sheth, P. S. (2021). VPN: Overview and security risks. *International Journal of Advanced Research in Science, Communication and Technology*, 7(1), 305-309.
- Sharpe, R., Warnicke, E., & Lamping, U. (n.d.). *Wireshark user's guide version 4.1.0*. Wireshark. https://www.wireshark.org/docs/wsug_html_chunked/
- Swyx. (2020a, February 23). *Networking essentials: Software defined networking*. DEV. <https://dev.to/swyx/networking-essentials-software-defined-networking-1631>
- Swyx. (2020b, February 23). *Networking essentials: Traffic engineering*. DEV. <https://dev.to/swyx/networking-essentials-traffic-engineering-2690>
- NetworkTutor. (2022, October 1). *Wireshark - Beginners guide - 101 | How to install and capture packets | How to filter ICMP | TCP* [Video].
- YouTube. <https://www.youtube.com/watch?v=Ud0QK0TPu4U>